SyncQuest

How Two People on Different Devices Use It

This guide covers the one scenario that needs a small extra setup step: two separate physical devices (not two tabs on the same laptop) joining the same realm over Wi-Fi. `localhost` only works within a single machine, so both devices must reach the host machine by its real network address. This edition also explains what the Part 2 features (coordinator election, vector clocks, audit log) look like when the peers are on different devices.

1 Run both servers on one machine (the host)

Pick one of the devices (or a third machine) to run the servers. From the `p2p-editor/` folder, in two separate terminals:

```
# Terminal 1 - signaling server (port 4444)
npm run signaling

# Terminal 2 - frontend (port 5173)
npm run dev
```

2 Find the host machine's LAN IP address

On Windows, open a terminal on the host and run `ipconfig`. Look for **IPv4 Address** under the active Wi-Fi adapter, for example `192.168.1.23`. Both devices must be on the same Wi-Fi network as the host.

> **The address changes.** A laptop's IP can differ between your home router and a phone hotspot, and changes whenever it reconnects. Re-run `ipconfig` each session instead of trusting a number from last time.

3 Point the frontend at that IP instead of localhost

Edit `frontend/.env` on the host (copy from `.env.example` if needed):

```
VITE_SIGNALING_URL=ws://192.168.1.23:4444
```

Use the real IP from Step 2, then **restart** `npm run dev` so Vite picks up the change. The dev server is configured with `host: true` in `vite.config.js`, so it accepts connections from other devices.

4 Open the app on device 1 and copy the Portal Link

On device 1 (it can be the host itself) open `http://192.168.1.23:5173`. Use the LAN IP, **not** `localhost`. A realm code is generated automatically. Click **Copy Portal Link** to get the shareable URL.

5 Open the same link on device 2

Send the link to device 2 (phone, another laptop, any browser on the same Wi-Fi) and open it. Both devices must be able to reach the host on ports **5173** (frontend) and **4444** (signaling).

6 Allow the firewall prompt on first run

If Windows asks whether to allow Node.js through the firewall, choose **Allow** for Private/Home networks. If it is denied, device 2's requests are silently blocked before they ever reach the app.

What you should see when it works

- Both devices show the **Portal Open - Synced!** status badge.
- The **Party panel** on each device lists the other device's avatar and name.
- Typing on either device appears on the other within about a second, and each device's cursor is shown in its own colour inside the other's editor.
- Both devices show the **same coordinator** in the crown badge under the header, usually within a couple of seconds of connecting.

Part 2 on two devices: a quick demo

What to do	What you should see
Open the link on both devices and wait ~2 s	The same name in the crown badge on both. It is the peer with the higher internal ID, so it is not predictable and can be either device.
Open **Debug panel** on both; type on both within about a second of each other	Vector Clocks tab: entries flagged pink as **concurrent**. If you type, wait 2 s, then type on the other device, they show before / after instead.
Click **Force re-election** on the non-coordinator	Election Log shows ELECTION and a response; the badge ends with the same coordinator.
Close the browser tab on the coordinator device	On the other device the badge briefly shows Electing coordinator... (or goes straight through it), then names that device itself as coordinator, with a YOU tag.
Open **Audit log** on the coordinator	Entries such as coordinator_elected, peer_joined and periodic checkpoints, each with a verified mark.

Closing a tab vs. losing the network. Closing a tab removes that device's presence almost immediately. If a device instead loses Wi-Fi or its battery dies, the other peers only notice after roughly 30 seconds (the Yjs awareness timeout), so re-election is correspondingly slower. Also, a device that joins *after* several audit entries exist may not receive the earlier entries (no backfill is implemented); the coordinator's own log is complete.

If it doesn't work

Most common cause: both devices are on the same Wi-Fi but the status stays on Connecting.... This is usually **router client isolation** (common on guest and public Wi-Fi), which blocks device-to-device traffic. It is a network setting, not an app bug. Try a home network or a personal mobile hotspot instead.

Symptom	Likely cause	Fix
Device 2 cannot load the page at all	Opened with localhost, or firewall blocked port 5173	Use the LAN-IP URL; allow Node.js on Private networks
Page loads but status stays Connecting...	Wrong or outdated VITE_SIGNALING_URL, signaling server down, or client isolation	Recheck the IP in .env, restart npm run dev, confirm the signaling terminal is running
Connection error after about 8 s	Signaling socket never connected	Check port 4444 is reachable from the device; check the firewall

Symptom	Likely cause	Fix
Connected, but different coordinators on each device	Election still settling, or a refresh mid-election	Wait a few seconds; click Force re-election on either device
Works on one network but not another	Symmetric NAT or strict firewall; there is no TURN server	Use the same simple Wi-Fi or a phone hotspot

- Double-check `VITE_SIGNALING_URL` uses the host's *current* LAN IP. A leftover address from an earlier session is the easiest mistake to make.
- Confirm the signaling server terminal is still running and shows no errors.
- Confirm both devices opened the app through the LAN-IP URL, not `localhost`.
- Same-machine tabs always sync through the browser's BroadcastChannel even if signaling is broken, so a working two-tab test does not prove the cross-device setup works.

See README.md for the full setup guide, deployment steps and testing checklist, and SyncQuest_Part2_Project_Report.pdf for the Part 2 features.